



Memory Management 3: Paging

Week 7: Perhitungan Alamat dengan Basis 2

informatika-itera.github.io/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Konsep Dasar Paging**
2. **Perhitungan Bit dan Bilangan Basis 2**
3. **Translasi Alamat Logis ke Fisik**
4. **Validasi dan Page Fault**
5. **Latihan Praktis**
6. **Ringkasan**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Memahami konsep dasar page, frame, dan page table.
2. Menghitung jumlah bit untuk page number, offset, dan frame number berdasarkan ukuran memori (basis 2).
3. Melakukan translasi alamat logis ke alamat fisik dengan rumus dasar.
4. Memvalidasi alamat logis dan mendeteksi page fault.
5. Menerapkan konsep paging pada sistem memori berskala besar (MB, GB).

Fokus Kelas Ini

Kita fokus pada **perhitungan biner** dan **translasi alamat** step-by-step dengan banyak latihan praktis.

Apa Itu Paging?

Teknik manajemen memori modern

Definisi

Paging adalah teknik yang memungkinkan alamat fisik memori tidak harus bersambungan (noncontiguous) saat mengalokasikan ruang untuk proses.

Tiga elemen kunci:

- **Page:** Unit memori logis (address space proses) berukuran tetap.
- **Frame:** Unit memori fisik (RAM) berukuran sama dengan page.
- **Page Table:** Struktur data yang memetakan page → frame.

Penting

Ukuran page dan frame *harus sama* untuk memastikan mapping berjalan benar!

Page vs Frame: Visualisasi

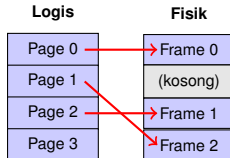
Memori logis vs memori fisik

Memori Logis (Proses)

- Program melihat memorinya linear dan bersambung.
- Dibagi menjadi *page* dengan ukuran tetap.
- Contoh: 4 KB/page, 8 KB/page.

Memori Fisik (RAM)

- RAM dibagi menjadi *frame* dengan ukuran sama.
- Frame dapat dialokasikan secara noncontiguous.
- Page dapat berada di mana saja di RAM.



Page Table: Mapping Page ke Frame

Struktur data penting dalam paging

Page Table Format

Page #	Frame #	Valid Bit
0	2	1
1	5	1
2	0	1
3	7	1
4	-	0

Isi Page Table

- Entry ke- i menyimpan frame number tempat page- i berada.
- Valid bit: 1 = page ada di memori, 0 = page tidak valid.
- OS dan MMU menggunakan ini setiap akses memori.

Konsep: Memerlukan Berapa Bit?

Hubungan antara ukuran dan jumlah bit

Rumus Dasar

Jika ada n item (page, frame, lokasi), maka memerlukan k **bit** di mana $2^k \geq n$.
Dengan kata lain: $k = \lceil \log_2 n \rceil$

Contoh:

- 4 page memerlukan $\lceil \log_2 4 \rceil = 2$ bit (karena $2^2 = 4$).
- 8 page memerlukan $\lceil \log_2 8 \rceil = 3$ bit (karena $2^3 = 8$).
- 256 byte page memerlukan $\lceil \log_2 256 \rceil = 8$ bit offset (karena $2^8 = 256$).

Tip Praktis

Offset (dalam bytes) = $2^k \Rightarrow$ memerlukan k bit offset

Tabel Referensi Basis 2

Standar yang sering muncul

Bit	Kombinasi	Byte/Item	Contoh
1 bit	2^1	2	2 item
2 bit	2^2	4	4 page
3 bit	2^3	8	8 frame
4 bit	2^4	16	16 byte
8 bit	2^8	256	256 byte (0.25 KB)
10 bit	2^{10}	1024	1 KB
12 bit	2^{12}	4096	4 KB (page standar)
20 bit	2^{20}	1.048.576	1 MB
30 bit	2^{30}	1.073.741.824	1 GB

Tangga Satuan Memory: 2^{10} Ladder

Hubungan antara Byte, KB, MB, GB, TB

Satuan	Nilai	Arti
1 B (Byte)	2^0	1 byte
1 KB (Kilobyte)	2^{10}	1024 byte
1 MB (Megabyte)	2^{20}	1024 KB = 1.048.576 byte
1 GB (Gigabyte)	2^{30}	1024 MB = 1.073.741.824 byte
1 TB (Terabyte)	2^{40}	1024 GB

Pola Kunci

Setiap level naik/turun, kalikan/bagi dengan $2^{10} = \mathbf{1024}$ (bukan 1000!).

- Naik: 1 MB = 1024 KB (kalikan dengan 1024)
- Turun: 1 MB = 1×2^{20} byte (kalikan dengan 2^{20})

Perkalian dalam Basis 2

Pangkat ditambahkan saat dikalikan

Aturan Dasar Eksponen

$$2^a \times 2^b = 2^{a+b}$$

Contoh 1: Konversi byte ke KB

- Diketahui: 2048 byte
- Hitungan: $2048 \div 2^{10} = 2^{11} \div 2^{10} = 2^{11-10} = 2^1 = 2 \text{ KB}$
- Atau: $2048 = 2 \times 1024 = 2 \text{ KB}$

Contoh 2: Konversi KB ke byte

- Diketahui: 8 KB
- Hitungan: $8 \times 2^{10} = 2^3 \times 2^{10} = 2^{3+10} = 2^{13} = 8192 \text{ byte}$

Contoh 3: Konversi MB ke byte

- Diketahui: 4 MB
- Hitungan: $4 \times 2^{20} = 2^2 \times 2^{20} = 2^{22} = 4.194.304 \text{ byte}$

Pembagian dalam Basis 2

Pangkat dikurangkan saat dibagi

Aturan Dasar Eksponen

$$\frac{2^a}{2^b} = 2^{a-b}$$

Contoh 1: Berapa banyak KB dalam 4096 byte?

- Hitungan: $\frac{4096}{1024} = \frac{2^{12}}{2^{10}} = 2^{12-10} = 2^2 = 4 \text{ KB}$

Contoh 2: Berapa banyak MB dalam 1 GB?

- Hitungan: $\frac{2^{30}}{2^{20}} = 2^{30-20} = 2^{10} = 1024 \text{ MB}$

Contoh 3: Berapa banyak frame 4 KB dalam 256 MB?

- $256 \text{ MB} = 2^8 \times 2^{20} = 2^{28} \text{ byte}$
- Jumlah frame = $\frac{2^{28}}{2^{12}} = 2^{28-12} = 2^{16} = 65.536 \text{ frame}$

Latihan: Konversi Satuan Memory

Gunakan pangkat 2 untuk mempermudah

Soal A

1. Berapa KB dalam 8192 byte?
2. Berapa byte dalam 16 MB?
3. Berapa MB dalam 2 GB?

Soal B

1. Berapa frame 8 KB dalam 512 MB?
2. Berapa byte dalam 256 page (setiap page 4 KB)?
3. Berapa KB dalam 64.000 byte (perkiraan)?

Solusi: Konversi Satuan Memory

Langkah-langkah penyelesaian

A1. 8192 byte ke KB

```
8192 = 213  
213 / 210  
= 23 = 8 KB
```

A2. 16 MB ke byte

```
16 MB  
= 24 * 220  
= 224  
= 16.777.216  
byte
```

A3. 2 GB ke MB

```
2 GB = 21 * 230  
231 / 220  
= 211  
= 2.048 MB
```

B1. Frame 8 KB dalam 512 MB

```
512 MB = 29*220  
8 KB = 23*210  
229 / 213  
= 216  
= 65.536  
frame
```

Struktur Alamat Logis

Cara membaca alamat virtual dalam paging

Setiap **alamat logis** terdiri dari dua bagian:

Page Number (p)	Offset (d)
------------------------	-------------------

- **Page Number (p):** Menunjukkan page mana yang diakses.
- **Offset (d):** Lokasi byte dalam page itu (0 sampai ukuran page - 1).

Contoh konkret: Sistem dengan 4 KB page (2^{12} byte):

- Offset memerlukan 12 bit (untuk 4 KB = 4096 byte).
- Jika ada 256 page, page number memerlukan 8 bit.
- Total alamat logis: $8 + 12 = \mathbf{20 \text{ bit}}$.

Mekanisme Translasi Alamat

Langkah-langkah hardware melakukan konversi

1. **Ekstrak** page number (p) dan offset (d) dari alamat logis.
2. **Cek** apakah page valid di page table (cek valid bit).
3. **Ambil** frame number (f) dari page table[p].
4. **Hitung** alamat fisik = $(f \times \text{ukuran page}) + d$.
5. **Akses** memori fisik pada alamat tersebut.

Rumus Translasi

$$\text{Alamat Fisik} = (\text{Frame Number} \times \text{Ukuran Page}) + \text{Offset}$$

Contoh Perhitungan 1: Sistem Kecil (5 bit)

Translasi alamat logis ke fisik

Spesifikasi Sistem

- Memori fisik: 32 byte (5 bit untuk offset penuh)
- Ukuran page: 4 byte (2^2)
- Total frame: $32 / 4 = 8$ frame (3 bit)
- Jumlah page: 8 page (3 bit)
- Alamat logis total: 5 bit (3 bit page + 2 bit offset)

Page Table:

Page 0 -> Frame 1
Page 1 -> Frame 4
Page 2 -> Frame 3
Page 3 -> Frame 7

Struktur Alamat (5 bit):

Page #	Offset
(3 bit)	(2 bit)

Contoh Perhitungan 1: Translasi Konkret

Alamat logis 13 → Alamat fisik ?

Langkah 1: Konversi Desimal ke Biner

Alamat logis 13 (desimal) = 01101 (biner)

Langkah 2: Ekstrak Page Number dan Offset

01101

^^^-- Page Number = 011 = 3 (desimal)

--^ Offset = 01 = 1 (desimal)

Langkah 3: Cari Frame Number di Page Table

Page 3 → Frame 7

Langkah 4: Hitung Alamat Fisik

Alamat Fisik = $(7 \times 4) + 1 = 28 + 1 = 29$

Contoh Perhitungan 2: Sistem Medium (7 bit)

Dengan page 16 byte

Spesifikasi Sistem

- Memori fisik: 128 byte
- Ukuran page: 16 byte (2^4) $\Rightarrow$ 4 bit offset
- Total frame: $128 / 16 = 8$ frame (3 bit)
- Jumlah page: ada 4 page (2 bit untuk page number)
- Alamat logis: 6 bit (minimal, tergantung jumlah page)

Page Table:

Page 0 -> Frame 2

Page 1 -> Frame 5

Page 2 -> Frame 0

Page 3 -> Frame 6

Dua Latihan:

1. Translasi alamat logis **22**?
2. Translasi alamat logis **53**?

Contoh Perhitungan 2: Solusi Latihan A

Alamat logis 22

Langkah per Langkah

1. Konversi ke biner: 22 (desimal) = 010110 (biner)

2. Ekstrak:

- Page number: 01 = 1 (desimal)
- Offset: 0110 = 6 (desimal)

3. Cari frame: Page 1 → Frame 5

4. Hitung fisik: $(5 \times 16) + 6 = 80 + 6 = 86$

Contoh Perhitungan 2: Solusi Latihan B

Alamat logis 53

Langkah per Langkah

1. Konversi ke biner: 53 (desimal) = 110101 (biner)

2. Ekstrak:

- Page number: 11 = 3 (desimal)
- Offset: 0101 = 5 (desimal)

3. Cari frame: Page 3 → Frame 6

4. Hitung fisik: $(6 \times 16) + 5 = 96 + 5 = 101$

Validasi Alamat Logis

Mendeteksi akses ilegal ke memori

Tidak semua alamat logis valid. Ada tiga kondisi error:

1. **Page Number Out of Range:** Page number $\geq$ jumlah page yang ada.
2. **Valid Bit = 0:** Page ada di page table tetapi belum dimuat ke RAM.
3. **Permission Violation:** Akses write ke page read-only, dll.

Page Fault

Ketika salah satu kondisi error terdeteksi, hardware mengirim **page fault exception** ke OS. OS kemudian memutuskan: load page, atau terminate proses.

Contoh: Validasi Alamat Logis 73

Dari contoh perhitungan 2

Data Sistem

- Jumlah page: 4 (page 0-3)
- Ukuran page: 16 byte (4 bit offset)

Analisis Alamat Logis 73

1. **Konversi:** 73 (desimal) = 1001001 (biner)

2. **Ekstrak:**

- Page number: 100 = **4** (desimal)
- Offset: 1001 = 9 (desimal)

3. **Validasi:** Page number 4 $\geq$ jumlah page (4)

4. **Hasil: TIDAK VALID** $\Rightarrow$ Page Fault!

Latihan 1: Sistem Basis 2 Sederhana

Kerjakan step-by-step

Soal

Diketahui sistem dengan spesifikasi:

- Ukuran memori fisik: 64 byte
- Ukuran page: 8 byte
- Page table:

Page	Frame
0	3
1	7
2	1

Tugas

1. Berapa bit untuk offset? Berapa bit untuk page number?
2. Translasikan alamat logis 19 ke alamat fisik.
3. Translasikan alamat logis 11 ke alamat fisik.

Solusi Latihan 1

Langkah demi langkah

1. Perhitungan Bit

- Ukuran page: 8 byte = $2^3 \Rightarrow$ **3 bit offset**
- Total frame: $64 / 8 = 8 \Rightarrow$ **3 bit frame**
- Total page: 3 page $\Rightarrow$ **2 bit page number**
- Alamat logis minimal: $2 + 3 = 5$ bit

2. Translasi Alamat Logis 19

```
19 (des) = 10011 (bin)
Page#: 10 = 2
Offset: 011 = 3
Page 2 -> Frame 1
Fisik = (1 * 8) + 3
= 11 [OK]
```

3. Translasi Alamat Logis 11

```
11 (des) = 01011 (bin)
Page#: 01 = 1
Offset: 011 = 3
Page 1 -> Frame 7
Fisik = (7 * 8) + 3
= 59 [OK]
```


Latihan 2: Sistem Heksadesimal (11 bit)

Dengan 256 byte page

Soal

Diketahui sistem dengan spesifikasi:

- Ukuran memori fisik: 4 KB (4096 byte)
- Ukuran page: 256 byte (2^8)
- Page table:

Page	Frame
0	5
1	9
2	14
3	3

Tugas

1. Hitung bit yang diperlukan untuk offset dan page number.
2. Translasikan alamat logis 0x2AF (heksadesimal) ke alamat fisik.
3. Translasikan alamat logis 0x15D (heksadesimal) ke alamat fisik.
4. Apakah alamat 0x6A1 valid? Mengapa?

Solusi Latihan 2 (Bagian 1-2)

Perhitungan bit dan translasi pertama

1. Perhitungan Bit

- Ukuran page: 256 byte = $2^8 \Rightarrow$ **8 bit offset**
- Total frame: $4096 / 256 = 16 \Rightarrow$ **4 bit frame**
- Total page: 4 page $\Rightarrow$ **2 bit page number**
- Alamat logis minimal: $2 + 8 =$ **10 bit**

2. Translasi Alamat Logis 0x2AF

```
0x2AF = 687 (des)
= 1010101111 (10-bit)
Page#: 10 = 2
Offset: 10101111 = 0xAF
= 175 (des)
Page 2 -> Frame 14
Fisik = (14 * 256)
+ 175 = 3759 [OK]
```

Solusi Latihan 2 (Bagian 3-4)

Translasi kedua dan validasi

3. Translasi Alamat Logis 0x15D

```
0x15D = 349 (des)
= 0101011101 (10-bit)
Page#: 01 = 1
Offset: 01011101
= 0x5D = 93 (des)
Page 1 -> Frame 9
Fisik = (9 * 256) + 93
= 2397 [OK]
```

4. Validasi Alamat Logis 0x6A1

```
0x6A1 = 1697 (des)
= 11010100001 (11-bit)
Page#: 110 = 6 (des)
```

TIDAK VALID!
Page 6 tidak ada
(hanya 0-3).

Hasil: PAGE FAULT [X]

Latihan 3: Sistem Skala Besar (32 bit)

Perhitungan dengan MB dan GB

Soal

Diketahui sistem dengan spesifikasi:

- Ruang alamat logis per proses: 256 MB (2^{28} byte)
- Memori fisik: 4 GB (2^{32} byte)
- Ukuran page: 4 KB (2^{12} byte)

Tugas

1. Hitung jumlah page per proses.
2. Hitung jumlah frame dalam sistem fisik.
3. Berapa bit untuk page number, offset, dan frame number?
4. Berapa byte untuk page table 1 proses (setiap entry 4 byte)?
5. Jika ada 100 proses, berapa MB untuk semua page table?

Solusi Latihan 3

Perhitungan sistem besar

1. Jumlah Page

256 MB / 4 KB
= $2^{28} / 2^{12}$
= 2^{16}
= 65,536

2. Jumlah Frame

4 GB / 4 KB
= $2^{32} / 2^{12}$
= 2^{20}
= 1,048,576

3. Bit untuk Alamat

Offset: 12 bit

Page num:
16 bit

Frame num:
20 bit

Logis: 28 bit
Fisik: 32 bit

4 & 5. Page Table

Per proses:
 $65,536 * 4 \text{ byte}$
= 262,144 byte
= 256 KB

100 proses:
 $100 * 256 \text{ KB}$
= 25,600 KB
= 25 MB

Checkpoint Pemahaman

Cek sebelum lanjut

1. Apa perbedaan antara page dan frame?
2. Kenapa ukuran page harus sama dengan ukuran frame?
3. Bagaimana cara menghitung jumlah bit offset dari ukuran page?
4. Diberikan alamat logis dalam biner, langkah apa untuk menerjemahkan ke alamat fisik?
5. Apa yang terjadi ketika page number melebihi jumlah page yang ada?

Jawaban Singkat

Jika Anda bisa menjawab semua dengan percaya diri, Anda siap untuk materi advanced memory management!

Ringkasan: 6 Poin Kunci

Intisari materi

1. **Paging** membagi memori logis menjadi page dan memori fisik menjadi frame, keduanya temsama ukuran.
2. **Offset** yang diperlukan = $\log_2$ (ukuran page dalam byte).
3. **Alamat logis** = (page number || offset) dalam biner.
4. **Page table** memetakan page number $\rightarrow$ frame number.
5. **Alamat fisik** = (frame number $\times$ ukuran page) + offset.
6. **Page fault** terjadi jika page number $\geq$ jumlah page yang ada.

Checklist Keterampilan

Pastikan Anda dapat melakukan ini

- ☐ Mengkonversi desimal $\leftrightarrow$ biner untuk alamat 5-32 bit.
- ☐ Menghitung jumlah bit offset dari ukuran page (dalam KB, MB, byte).
- ☐ Mengekstrak page number dan offset dari alamat logis biner.
- ☐ Mencari frame number di page table dan menghitung alamat fisik.
- ☐ Memvalidasi apakah alamat logis legal atau menyebabkan page fault.
- ☐ Menghitung ukuran page table untuk sistem dengan page count besar.

Terima Kasih

Ada Pertanyaan?

Jika ada satu langkah belum paham, jangan lanjut. Ulang checkpoint tadi, atau tanya instruktur.